Project Report — Recommendation Systems for Personalized Content Discovery

Project & Implementation: Priyanshu Nigam
Report author: Priyanshu Nigam
Date: 2026-06-11
Dataset: Netflix Prize (≈100M ratings, ~480K users, ~17.7K movies)

1. Executive Summary

The project delivers an end-to-end recommendation system on the Netflix Prize dataset, taking raw text-dump rating logs all the way to an interactive, explainable web dashboard. Rather than chasing a single complex model, the work follows a deliberate progression — **global baselines** → **neighborhood collaborative filtering** → **matrix factorization** → **neural CF** → **a hybrid ensemble** — and layers a **post-hoc explanation engine** and a **Streamlit demo** on top.

The system is evaluated on two complementary axes that matter for real recommenders: **rating-error accuracy** (RMSE) and **top-K ranking relevance** (MAP, Precision, Recall, NDCG, Hit Rate, Coverage). The headline configuration is a weighted ensemble (70% Funk SVD + 30% item-based CF) reaching an RMSE of ≈0.80 on a dense evaluation slice.

The codebase is clean, modular, and reproducible in structure (config-driven pipeline, one script per modeling phase). This report documents what was built, **justifies the design decisions**, and gives an honest engineering critique of the evaluation methodology and the gaps that a production deployment would need to close.

2. Data & Exploratory Analysis

2.1 Raw data

The source is the classic Netflix Prize release: four `combined_data_*.txt` files (~2 GB total) in a custom format where a movie header line (`movieId:`) is followed by `customerId,rating,date` rows. Side files include `movie_titles.csv` (id, year, title) and the official `probe`/`qualifying` splits.

2.2 Ingestion pipeline (src/data/make_dataset.py)

The parser streams each text file line-by-line (constant memory), tracks the "current movie" header, and emits a columnar table written to **Parquet**. Ratings are stored as `UInt8` and dates parsed to a `Date` dtype.

> **Justification — why Parquet + Polars:** the raw text is ~2 GB and ~100M rows. Parquet gives columnar compression (the 2 GB of text compresses to ~525 MB) and lazy/streaming reads, and Polars' lazy engine (`scan_parquet` / `sink_parquet`) lets feature aggregation run without materializing the full frame in RAM. This is the right call for laptop-scale work on a dataset this large — it is the difference between "runs" and "OOM."

2.3 EDA (src/data/runeda.py, notebooks/01exploratorydataanalysis.ipynb)

The EDA quantifies the two properties that drive every downstream modeling decision:

- **Extreme sparsity.** With ~480K users × ~17.7K movies, the user–item matrix is >99% empty. This rules out dense matrix methods and motivates latent-factor and neighborhood models.
- **The long tail.** Movie popularity is plotted on a log scale and user activity as a log-binned histogram — both confirm a heavy-tailed distribution: a few blockbusters and power-users dominate, while most items/users have very few interactions.

> **Engineering read:** the long tail is *why* the project later evaluates **Coverage** (are we only recommending blockbusters?) and *why* a dense top-users/top-movies slice is used for the expensive models — the cold tail simply doesn't have enough signal for KNN/MF to learn from.

3. Feature Engineering

Feature work (src/data/build_features.py) is intentionally lean and produces three processed artifacts:

Artifact	Contents
`interactions.parquet`	Cleaned (user, item, rating, date) event log — the modeling substrate
`user_features.parquet`	Per-user rating **count** and **mean rating**
`movie_features.parquet`	Per-movie rating **count** and **mean rating**, joined to title/year metadata

> **Justification — why so few features:** collaborative filtering and matrix factorization learn latent user/item representations *directly from the interaction matrix*; they do not consume hand-crafted features. The engineered aggregates (counts, means) are exactly what the downstream models actually need: means feed the **bias baseline** and `KNNWithMeans` mean-centering, and counts drive the **popularity slicing** used to build dense evaluation subsets. Adding genre/temporal features would only pay off for a content-based or feature-augmented hybrid, which is correctly scoped out here. This matches Guideline #3 (favor simplicity with strong reasoning).

A reasonable extension (see §9) would be **temporal features** (rating recency, user tenure) and **release-year/decade** signals, which the metadata join already makes available.

4. Modeling Approach

The project's strongest quality is its **laddered model progression** — each tier is a meaningful baseline for the next, which directly satisfies Guideline #4 (compare multiple approaches) and Guideline #2 (justify model choices).

4.1 Tier 1 — Global baselines (src/models/baselines.py)

Four reference points of increasing sophistication:

1. **Global Mean** — predict μ for everyone (sanity floor).
2. **User Mean** — captures per-user rating leniency.
3. **Movie Mean** — captures per-item quality.
4. **Regularized Bias Baseline** — `r■_ui = μ + b_u + b_i` with shrinkage (`reg_i=25`, `reg_u=10`) so low-count items/users are pulled toward the global mean, and predictions are clipped to [1, 5].

> **Justification:** the bias baseline is the *correct* yard-stick for a rating predictor. Damped bias terms are the same trick used in the SVD++ family, and any latent model that can't beat it is not learning interaction signal. Computing it in pure Polars group-bys keeps it fast over the full sample.

4.2 Tier 2 — Neighborhood CF (src/models/collaborative_filtering.py)

`KNNWithMeans` (Surprise) with **Pearson-baseline similarity**, `k=40`, `min_support=5`, in both **item-based** and **user-based** modes.

> **Justification:** Pearson-baseline removes user/item bias *before* measuring similarity (more robust than raw cosine on rating data); `min_support=5` suppresses spurious neighbors from items co-rated by only one or two users. Item-based is the deployment default because item–item similarities are stable, precomputable, and — crucially — **directly explainable** ("because you liked X"), which the dashboard exploits.

4.3 Tier 3 — Matrix factorization (src/models/matrix_factorization.py)

Two latent-factor models for two different objectives:

- **Funk SVD** (Surprise, 100 factors, 20 epochs, biased) — optimizes regularized squared error on *observed explicit ratings*. This is the workhorse for RMSE.
- **Implicit ALS** (`implicit` library, 100 factors) — optimizes a confidence-weighted *ranking/implicit* objective. The code honestly comments that ALS scores are not calibrated to the 1–5 scale and clips the raw dot product into range.

> **Justification:** including both is exactly the "think like an ML engineer" point (Guideline #5) — explicit-rating SVD and implicit-feedback ALS answer different questions (how *highly* will you rate this vs. how *likely* are you to engage). Keeping them separate, and noting ALS's scale mismatch in code, shows awareness that a single RMSE number can be misleading across objectives.

4.4 Tier 4 — Neural CF (src/models/ncf.py)

A PyTorch NCF: user/item embeddings (dim 32) concatenated and passed through an MLP (`64 → 32 → 16`, ReLU, dropout 0.2) to a scalar rating. Trained with MSE/Adam for 5 epochs on the dense subset.

> **Justification & honest note:** NCF tests whether a non-linear interaction model beats the linear dot-product of SVD. As implemented it is a solid proof-of-concept but **under-tuned for a fair fight**: only 5 epochs, no output activation scaling to the rating range, and no early stopping / validation-loss tracking. It demonstrates capability (Guideline #4) more than it delivers a tuned production candidate — which the project implicitly acknowledges by *not* selecting it for the ensemble.

4.5 Tier 5 — Hybrid ensemble (src/models/hybrid.py)

A `WeightedEnsemble` blending **Funk SVD (0.70) + item-based CF (0.30)**, with weight-sum and length assertions and final clipping to [1, 5].

> **Justification — the central modeling decision:** SVD and item-CF make **uncorrelated errors** — SVD captures global latent structure, item-CF captures local neighborhood structure. Blending uncorrelated predictors is the most reliable way to reduce variance, and it is *exactly* the family of approach that won the original Netflix Prize. The 70/30 split sensibly weights the stronger-RMSE model (SVD) higher while retaining item-CF's complementary signal and its explainability hook. This is a textbook performance-vs-simplicity trade-off (Guideline #3).

---

5. Recommendation Strategy & Explainability

The project correctly internalizes Guideline #1 — **the deliverable is good recommendations, not just a low error number.**

5.1 Top-N generation

In the demo path (scripts/run_explain.py, app.py): for a target user, all catalog candidates **the user hasn't already seen** are scored, sorted by predicted rating, and the top-N surfaced. Already-watched items are filtered out — a small but essential product detail.

5.2 Explanation engine (src/models/explainability.py)

A **post-hoc, item-based justification generator**. For a recommended item it inspects the user's liked history (rating ≥ 4), finds the historically-liked item with the highest item–item similarity to the recommendation, and renders natural language:

> *"We recommend 'X' because you previously enjoyed 'Y' (Similarity Score: 0.89)."*

It degrades gracefully with tiered fallbacks: unknown item → "broadly popular"; cold-start user → "general popularity (new user)"; weak similarity (≤ 0.1) → "broader community trends."

> **Justification:** explanations are computed from the **same item-CF similarity matrix** that produced the recommendation, so the rationale is *faithful* to the model rather than a fabricated story. The graceful fallbacks show real cold-start awareness. Choosing item-CF (not SVD) as the explanation source is deliberate: latent SVD factors are not human-interpretable, whereas "similar to a movie you liked" is. This is why item-CF earns its place in the ensemble beyond raw accuracy.

5.3 Deployment surface (app.py)

A Streamlit dashboard: select a user, view their watch history, and get top-N recommendations with an expandable "Why this movie?" explanation per item. Model training is cached (`@st.cache_resource`) on a dense subset for ~15s cold start, then interactive.

> **Engineering read:** this closes the loop from data to *user-facing product* and demonstrates the "practical deployment considerations" the guidelines ask for. It is a demo-grade surface (trains on load over a 500×2000 slice rather than serving a precomputed model), which is appropriate for the assignment scope.

---

6. Evaluation Methodology & Metrics

Evaluation lives in src/evaluation/metrics.py and is run from scripts/run_evaluation.py.

6.1 Metric suite — and why each is present

Metric	What it measures	Why it matters here
RMSE	Rating-prediction error	Direct accuracy; comparable to Netflix Prize literature
MAP@10	Average precision over ranked list	Rewards putting relevant items *early*
Precision@10	Relevant share of top-10	"Is the shortlist good?"
Recall@10	Captured share of all relevant	"Did we miss good items?"
NDCG@10	Position-discounted relevance	Best single ranking-quality summary
Hit Rate	≥1 relevant item in top-10	User-perceived "did it work at all"
Coverage	Catalog fraction ever recommended	Guards against blockbuster-only collapse (the long tail)

> **Justification:** pairing **error metrics with ranking metrics** is the most important evaluation decision in the project and aligns squarely with Guideline #1. A recommender can have great RMSE and still produce a boring or repetitive list; Coverage + NDCG + MAP catch that. Implementing the ranking metrics as vectorized Polars window functions (cumulative-relevant / rank) keeps them fast over the validation set. Relevance is defined as rating ≥ 4, a standard explicit-feedback threshold.

6.2 Reported scorecard

(From README.md; the comprehensive scorecard is produced by `run_evaluation.py`.)

Metric	Score
RMSE	0.8049
MAP@10	0.8569
Precision@10	0.8849
Recall@10	0.2272
NDCG@10	0.9014
Hit Rate	0.9980
Coverage	0.9460

Reading the numbers: RMSE ≈0.80 is genuinely strong (it would have been competitive on the original Prize leaderboard). High Precision/NDCG/Hit Rate with **low Recall (0.23)** is the expected signature of a top-10 cutoff against users who have many relevant items — you can only fit 10 hits in a list, so recall is capped while precision stays high. Coverage 0.95 confirms the recommender spreads across the catalog rather than collapsing onto blockbusters.

---

7. Critical Engineering Assessment

In the spirit of Guideline #5 (think like an ML engineer) and #6 (reproducibility), here is the honest critique a reviewer should raise before this goes anywhere near production.

7.1 Evaluation caveats (the most important section)

1. **Ranking is scored over observed ratings only, not the full catalog.** The ranking metrics rank each user's *held-out rated items* by predicted score. Real top-N serving must rank **all unseen items** (mostly negatives). Evaluating only over items the user actually rated **inflates Precision/NDCG/Hit Rate** substantially — these are best read as *relative* model-comparison numbers on a favorable slice, **not** as production retrieval quality. A negative-sampling or all-unseen-candidate protocol would give more trustworthy ranking figures.
2. **Headline scorecard model labeling.** `run_evaluation.py` builds the comprehensive scorecard from the **Funk SVD** model, while the README attributes RMSE 0.8049 to the *Ensemble*. The ensemble RMSE is produced separately by `run_hybrid.py`. The two should be reconciled / clearly labeled so readers know exactly which model each number describes.
3. **Random (not temporal) split.** Splits are random interaction hold-outs. Netflix has real dates; a **time-based split** (train on past, predict future) would better reflect deployment and avoid look-ahead leakage.
4. **Dense-slice evaluation.** Expensive models are evaluated on the top-2000-users × top-500-movies slice. This is a defensible compute trade-off, but results **do not characterize cold/tail performance**, which is precisely where recommenders struggle. This limitation should be stated alongside the scorecard.

7.2 Reproducibility gaps (quick, high-value fixes)

- README references `scripts/run_parser.py` (the actual entry point is src/data/make_dataset.py) and `pip install -r requirements.txt` (there is **no** `requirements.txt`; dependencies live in pyproject.toml — install with `pip install -e .`). These doc/repo mismatches would block a clean first run.
- `pyproject.toml` author is a placeholder ("Principal ML Engineer / engineer@example.com") rather than the actual author.
- `dev` dependencies include `pytest`, but there is **no test suite**. Even a handful of unit tests on the metric functions (which are subtle window-function logic) would materially raise confidence.
- `wandb` is a declared dependency but experiment tracking is not wired into the run scripts; results are printed to stdout rather than logged/persisted to a scorecard file.

7.3 Model-level notes

- **NCF** is under-trained (5 epochs, no rating-range output scaling, no early stopping) — fine as an experiment, not yet a fair competitor.
- **Implicit ALS** scores are uncalibrated to 1–5 (acknowledged in code); comparing its RMSE directly to explicit models is apples-to-oranges and should be framed as ranking-oriented.

---

8. Strengths Summary

- **Clear, justified model ladder** from trivial baselines to a principled hybrid — every tier earns its place and the ensemble choice is well-reasoned (uncorrelated-error blending).
- **Dual-axis evaluation** (error *and* ranking, including Coverage) that takes recommendation *quality* seriously rather than optimizing a single proxy.

- **Faithful, graceful explainability** derived from the actual model, plus a working interactive demo — the project ships a *product*, not just a notebook.
- **Scalable, config-driven data engineering** (Polars + Parquet, lazy/streaming) appropriate to a 100M-row dataset on commodity hardware.
- **Clean repository structure** (`src/` package, one script per phase, central config) that is easy to read and extend.

---

9. Recommended Next Steps

1. **Fix the ranking evaluation protocol** — rank against all unseen items (or sampled negatives) to get trustworthy production-grade Precision/Recall/NDCG.
2. **Adopt a temporal split** to remove look-ahead leakage and report cold-start/tail metrics.
3. **Reconcile README ↔ scripts** — correct the parser/requirements references, add a `requirements.txt` or document `pip install -e .`, and label exactly which model each scorecard number belongs to.
4. **Persist results & wire up W&B** — write the scorecard to a versioned file and log runs, so model comparisons are reproducible rather than transient stdout.
5. **Add unit tests** for the metric functions (RMSE, MAP@K, NDCG) with hand-checked toy inputs.
6. **Tune NCF fairly** (more epochs, sigmoid-scaled output, early stopping) before declaring a linear-vs-neural verdict.
7. **Feature-augmented hybrid** — fold in temporal recency and release-year/genre signals to address cold-start, using the metadata join that already exists.

---

10. Conclusion

The project is a well-engineered, honest, and genuinely *useful* recommendation system that does the right things in the right order: it treats sparsity and the long tail as first-class problems, builds a justified model ladder culminating in a complementary hybrid, evaluates on both error and ranking quality, and ships explainable, user-facing recommendations. Its main weaknesses are in **evaluation rigor** (ranking measured on a favorable observed-item slice) and **documentation/reproducibility hygiene** (README/dependency drift, no persisted results or tests) — all of which are addressable without re-architecting anything. Measured against the guidelines, it scores highly on recommendation quality, justified decisions, the performance/simplicity balance, and experimentation; the clearest upside remaining is tightening evaluation methodology and reproducibility.